

CH4 - Clip 13.

버전관리



01

버전 관리란?

02

코드 버전 관리

03

데이터 버전 관리

1. 버전관리란?

버전 관리

정의

- 소프트웨어의 모든 구성 요소(코드, 데이터, 환경 설정 등)에 대한 변경 사항을 추적하고 관리하는 프로세스
- 프로젝트의 재현성, 안정성 및 협업 효율성을 높이는 데 매우 중요

MLOps에서의 버전 관리

- MLOps에서는 코드, 데이터, 모델, 환경 설정 등에 대한 변경 사항을 추적하고 관리하는 프로세스로 확장

MLOps에서의 버전 관리

코드 버전 관리

- 소스 코드의 변경 사항을 추적
- Git과 같은 도구를 사용하여 브랜치, 병합, 커밋 등을 관리

데이터 버전 관리

- 데이터 세트의 버전을 관리
- 데이터 변경 사항, 업데이트, 변형 과정을 추적

MLOps에서의 버전 관리

모델 버전 관리

- 모델의 각 버전을 추적
- 훈련된 모델, 파라미터, 하이퍼 파라미터 등의 버전을 관리

환경 및 구성 버전 관리

- 머신 러닝 모델을 훈련하고 실행하는 데 사용되는 환경의 버전을 관리
- Docker와 같은 컨테이너화 도구를 사용하여 환경을 일관되게 유지

MLOps에서의 버전 관리 중요성

재현성

- 실험 결과로 정확하게 재현할 수 있어야 함
- 데이터, 모델, 코드의 버전 관리를 통해 언제든지 특정 시점의 실험을 재현할 수 있어야함

협업

- 다수의 데이터 과학자와 엔지니어가 동일한 프로젝트에 기여할 때 일관성을 유지
- 변경 사항을 추적하고 충돌을 방지하여 팀 간의 효율적인 협업을 지원

MLOps에서의 버전 관리 중요성

품질 관리

- 버전 관리를 통해 변경 사항에 대한 품질 검증을 수행할 수 있음
- 코드 리뷰, 테스트, CI/CD 파이프라인을 통해 높은 품질의 모델을 보장

규정 준수 및 감사

- 머신러닝 프로젝트의 각 단계를 문서화하고 기록을 유지
- 규정 준수 요구 사항을 충족하고, 필요 시 감사를 용이하게 함

2. 코드 버전 관리

Git

정의

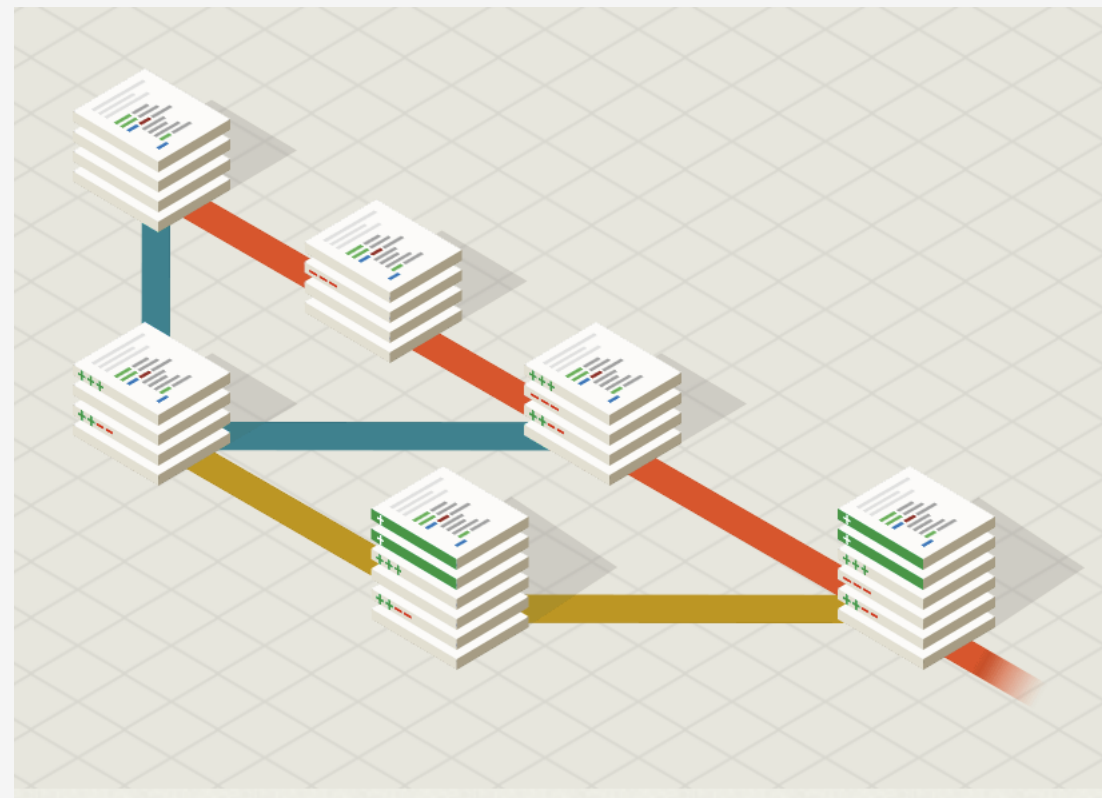
- 분산 버전 관리 시스템으로 소프트웨어 개발에서 소스 코드의 변경 사항을 추적하는데

기능

- **Branching & Merging:** 기능 개발이나 버그 수정을 위한 브랜치 생성 및 병합 지원
- **Change Tracking:** 코드 변경 사항 추적 및 이력 관리
- **Collaboration:** 다수의 개발자가 동일한 프로젝트에 기여할 수 있는 협업 기능

MLOps 적용

- 모델 개발 및 실험 단계에서의 코드 변경 관리
- 다양한 실험의 버전을 관리하고 최적의 모델을 선택하는 데 도움



GitHub/GitLab

정의

- Git repository 호스팅 서비스로 코드 공유, 협업 및 버전 관리 기능을 제공

기능

- **Issue Tracking:** 작업 항목 및 버그 추적
- **Pull Requests/Merge Requests:** 코드 리뷰 및 변경 사항 통합
- **CI/CD Integration:** 지속적 통합 및 배포 파이프라인과의 통합

MLOps 적용

- 코드 리뷰를 통한 품질 관리
- CI/CD 파이프라인을 통한 자동화된 모델 훈련 및 배포



3. 데이터 버전 관리

데이터 버전 관리

정의

- 머신러닝 프로젝트의 데이터 세트의 버전을 관리하는 역할
- 데이터의 변경 사항을 추적하고, 여러 버전의 데이터 세트를 효율적으로 관리할 수 있도록 도와줌

종류

- DVC
- Git LFS
- Delta Lake

DVC

정의

- 오픈 소스 버전 관리 시스템
- Git과 유사한 인터페이스를 제공하지만, 대용량 데이터 파일/데이터 세트/머신러닝 모델 등을 효율적으로 관리하는데 특화

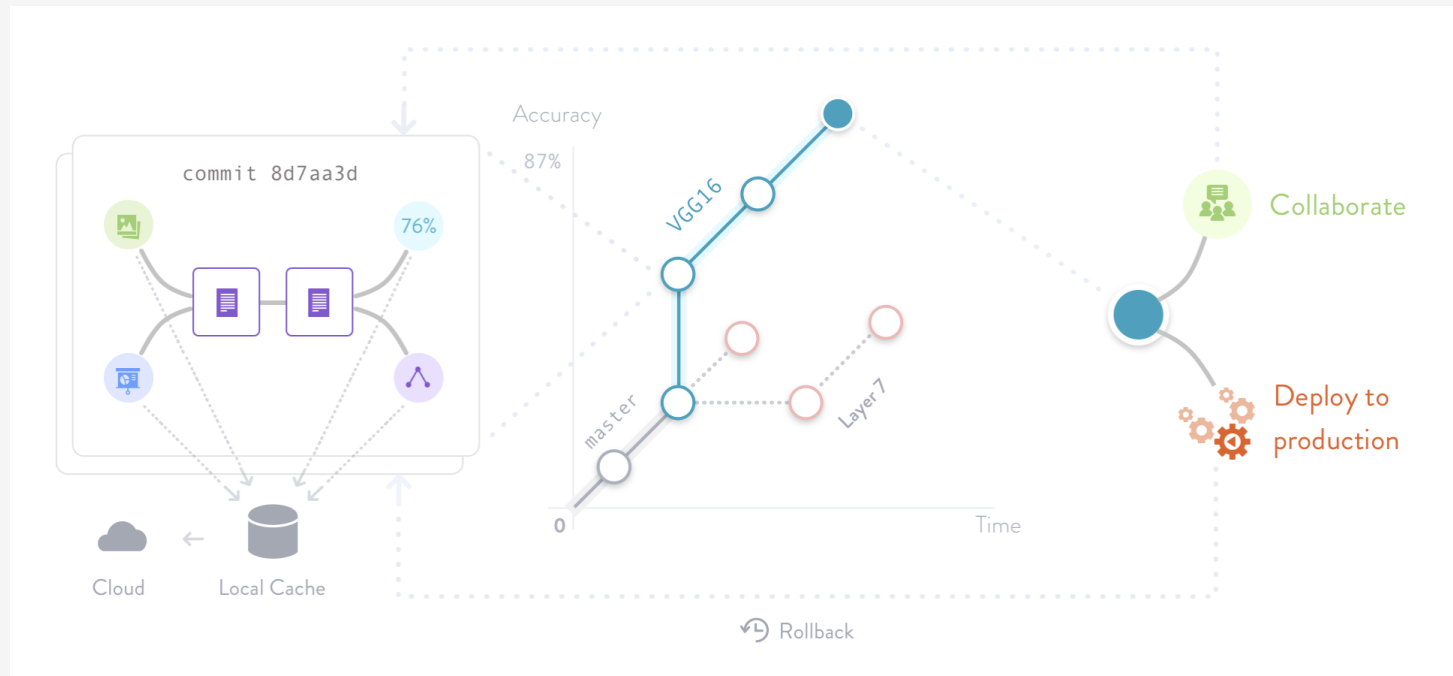
주요 기능

- **대용량 데이터 관리** : DVC는 Git의 한계를 극복하고 대용량 데이터 파일을 효율적으로 관리. 데이터 파일은 원격 저장소에 저장되며, 로컬에는 파일의 메타데이터만 유지
- **데이터 세트 버전 관리** : 데이터 세트의 버전을 Git과 유사한 방식으로 관리할 수 있음. 이를 통해 데이터 세트의 변경 사항을 추적하고, 필요에 따라 이전 버전으로 쉽게 되돌릴 수 있음
- **데이터 파이프라인 관리** : 데이터 처리 및 머신러닝 모델 훈련을 위한 파이프라인을 정의하고 버전 관리할 수 있음. 이를 통해 전체 워크플로우의 재현성과 투명성을 보장
- **실험 관리 및 추적** : 다양한 실험에 사용된 데이터 세트, 하이퍼파라미터, 모델 등을 추적. 실험 결과를 비교하고, 최적의 결과를 쉽게 식별할 수 있음
- **원격 저장소 지원** : AWS S3, Google Cloud Storage, Azure Blob Storage 등 다양한 클라우드 스토리지 서비스를 원격 저장소로 사용할 수 있음. 또한, SSH, NFS 등을 통한 원격 접근도 지원.

DVC

특징

- **Git 과의 통합** : DVC는 Git과 밀접하게 통합되어 있어, Git을 사용하는 사용자에게 친숙한 명령어와 워크플로우를 제공
- **확장성** : 대규모 데이터 세트와 복잡한 데이터 파이프라인을 효율적으로 관리할 수 있도록 설계
- **플랫폼 독립성** : DVC는 다양한 운영 체제에서 사용할 수 있으며, 다양한 클라우드 스토리지 옵션을 지원
- **커뮤니티 및 문서** : 활발한 커뮤니티와 잘 정리된 문서를 통해 사용자가 쉽게 시작할 수 있고, 문제 해결에 도움을 받을 수 있음



Git LFS

정의

- Git의 확장 도구로서 대용량 파일을 효율적으로 관리
- Git은 소스 코드와 같은 작은 파일을 관리하는데 최적화되어 있지만, 대용량 파일에는 제한이 있기 때문에 Git LFS가 설계됨

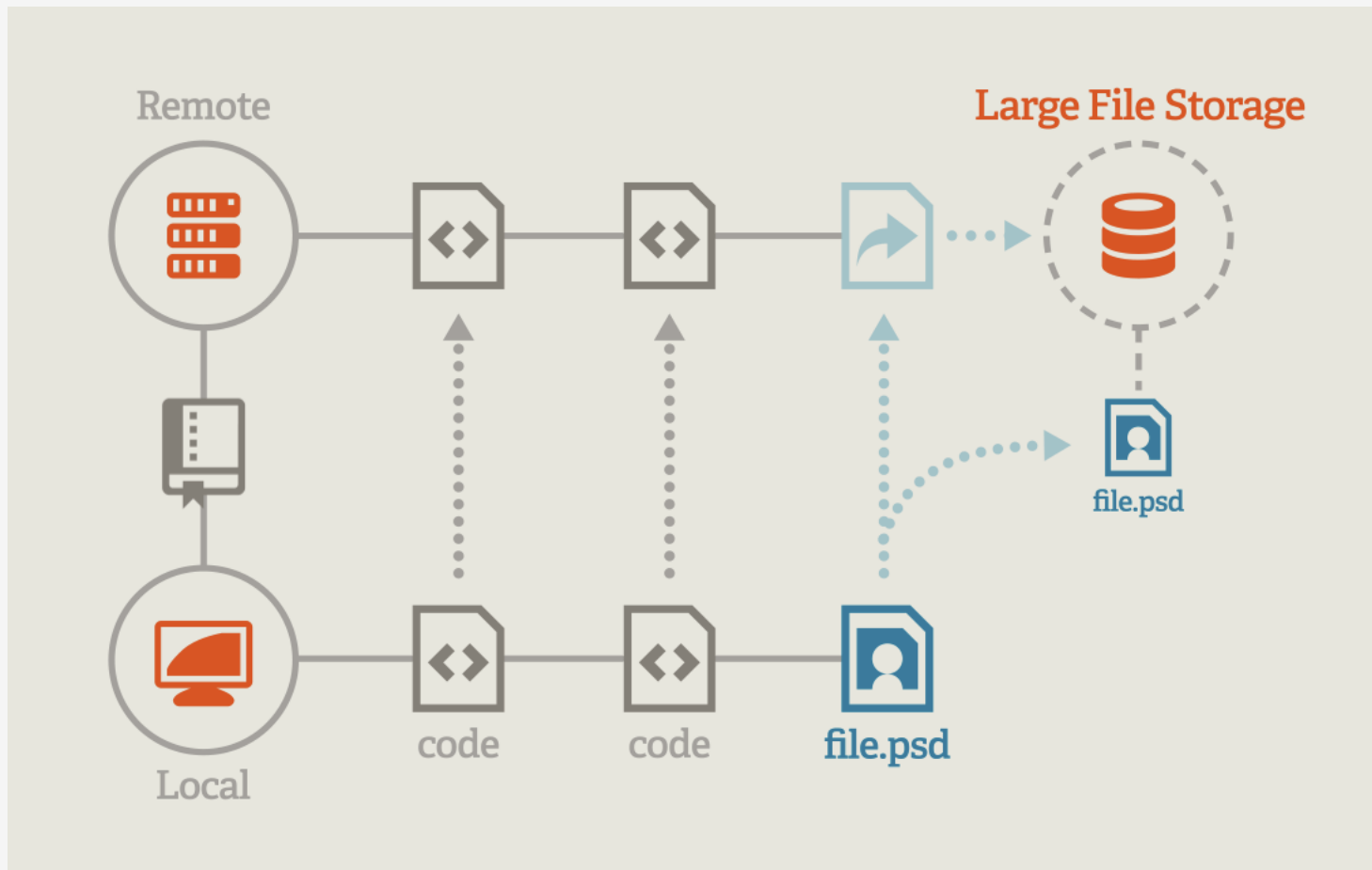
주요 기능

- **대용량 파일 관리** :Git LFS는 대용량 파일에 대한 포인터를 Git 리포지토리에 저장하고, 실제 파일은 별도의 LFS 서버에 저장. 이를 통해 리포지토리의 크기를 줄이고 성능을 최적화
- **버전 관리** : Git LFS를 사용하면 대용량 파일도 Git의 버전 관리 체계 하에 놓일 수 있음. 이는 파일의 변경 이력을 추적하고, 필요에 따라 이전 버전으로 쉽게 되돌릴 수 있음을 의미
- **효율적인 데이터 전송** : Git LFS는 필요한 파일만 선택적으로 다운로드하거나 업로드할 수 있게 해줌. 이는 네트워크 대역폭과 저장 공간을 효율적으로 사용할 수 있음
- **통합 및 호환성** : Git LFS는 Git과 밀접하게 통합되어 있어, 기존의 Git 명령어와 유사한 방식으로 사용할 수 있음. 대부분의 Git 호스팅 서비스에서 지원

Git LFS

특징

- **확장성** : 대용량 파일을 효과적으로 관리함으로써, Git 리포지토리의 확장성을 크게 향상
- **성능 최적화**: 대용량 파일을 별도로 관리함으로써, Git 리포지토리의 클론 및 풀 작업 시의 성능 저하를 방지
- **유연한 작업 흐름** : 대용량 파일을 위한 별도의 저장소를 사용함으로써, 개발자는 소스 코드와 대용량 파일을 별도로 관리할 수 있어 작업 흐름이 더 유연해짐
- **쉬운 설정 및 사용** : .gitattributes 파일을 통해 LFS로 관리할 파일 유형을 쉽게 지정할 수 있으며, 일반적인 Git 명령어를 그대로 사용할 수 있음



Delta Lake

정의

- Apache Spark 위에 구축된 오픈 소스 저장소 레이어
- 대규모 Data Lake 환경에서 데이터의 신뢰성과 성능을 향상시키기 위해 설계됨
- ACID 트랜잭션을 지원하며, 데이터 무결성과 복잡한 데이터 파이프라인 관리를 용이하게 함

주요 기능

- **ACID 트랜잭션** : ACID(Atomicity, Consistency, Isolation, Durability) 트랜잭션을 제공하여 데이터의 무결성을 보장. 이는 데이터 레이크에서 발생할 수 있는 동시성 문제와 데이터 손상 문제를 해결
- **스케일러블 메타데이터 처리** : 대규모 데이터 세트의 메타데이터를 효율적으로 처리할 수 있도록 설계. 이는 데이터 레이크의 성능과 확장성을 크게 향상
- **Time Travel** : 데이터의 이전 버전으로 쉽게 되돌아갈 수 있는 '시간 여행' 기능을 제공. 이를 통해 데이터 변경 이력을 추적하고, 필요에 따라 이전 상태로 데이터를 복원할 수 있음
- **통합 스트리밍 및 배치 처리** : Delta Lake은 스트리밍 데이터와 배치 데이터를 동일한 프레임워크에서 처리할 수 있도록 지원. 이는 복잡한 데이터 파이프라인을 단순화하고, 실시간 데이터 처리를 가능
- **스키마 검증** : 데이터 스키마의 검증 및 자동 진화를 지원

Delta Lake

특징

- **데이터 무결성 및 신뢰성** : ACID 트랜잭션과 스키마 검증 기능을 통해 데이터 무결성과 신뢰성을 보장
- **성능 최적화**: 스케일러블 메타데이터 처리와 효율적인 파일 관리를 통해 대규모 데이터 세트의 처리 성능을 최적화
- **유연한 데이터 처리** : 스트리밍 및 배치 데이터 처리의 통합은 다양한 유형의 데이터 처리 요구 사항을 유연하게 충족
- **Spark 통합** : Apache Spark와의 긴밀한 통합은 Spark 사용자에게 친숙한 환경에서 데이터 레이크를 관리할 수 있게함

